

■ 4.1.5 Data Types

Many programming languages make you declare the “types” of all the variables or functions you use. There are several reasons for this. Historically, the main reason was that operations on integers could usually be performed much more efficiently on standard computer hardware than the corresponding operations on floating point numbers. As a result, it was important for you to specify whether a particular variable could be taken to be an integer, and therefore manipulated with more efficient operations.

Another reason to use data types is to allow you to understand and check the structure of your program more easily. If a particular variable is supposed to be of a particular type, and you try to use it in a way that is inconsistent with its type, it is worthwhile for the programming language to tell you. This is especially valuable for *compiled* languages such as C and Pascal where such errors can be recognized at the compilation stage.

The major disadvantage of requiring you to declare the types of all your variables is that any program you write must start with lines of declarations which, in and of themselves, perform no useful operations.

At a fundamental level, there are no data types in *Mathematica*. Every object you use is an *expression*, and every function can take any expression as an argument. Of course, there may only be rules for the value of a function when its arguments have a particular form. If the arguments do not have that form, the function is left unevaluated.

Mathematica allows you to check the form of arguments to functions in several ways. At the simplest level, you can define a function by giving a transformation rule for a pattern that involves something like `n_Integer`. This object only *matches* expressions that are integers. As a result, you can think of the variable `n` in this particular pattern as being required to be of “type” `Integer`.

You can also use `?` and `/;` to specify constraints on the arguments to functions. You need not just say that `n` must be an integer; you can use `?` or `/;` to specify explicitly that it must be an integer, say, between 0 and 5.

It is often convenient to create objects that act like “packets” of data. Lists are probably the most common example. The only role of the *Mathematica* function `List` is to “tag” a collection of objects. You can think of this “tag”, as specifying that the collection is of “type” `List`. There are then many *Mathematica* functions which act only on this particular type of data. The functions are defined by giving transformation rules for patterns in which their arguments are lists. No transformation rules are given when their arguments are not lists, and so no values can be found in these cases.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

You can easily create your own “types” of data in *Mathematica*. For example, if you want to create a type that represents an integer modulo n , you can just start using objects of the form `ModularInteger[i, n]`. The function `ModularInteger` itself may do nothing. But you can use it as a “tag” to indicate to other functions that the object you are using is of “type” `ModularInteger`. You can then define functions that act on `ModularInteger` objects just by specifying rules in which the arguments of functions are patterns involving `ModularInteger`. So, for example, you could give a rule for `negate[ModularInteger[i_, n_]]` to specify how to negate modular integers.